

Stanford CS336: Language Modeling from Scratch

后训练与可验证奖励强化学习 (RLVR)



视频 ID: dIFAi87Ws4E 时长: 01:15:50 发布日期: 2026-05-27

频道: Stanford Online 视频: <https://youtu.be/dIFAi87Ws4E>

阅读方式: 本笔记将课程概念、公式、算法和工程边界重组为可独立阅读的教材。

目录

1 导读：RLVR 为什么成为后训练主线	3
1.1 RLHF 与 RLVR 的共同骨架	4
1.2 本章小结	4
2 PPO 回顾：理论目标与语言模型实现	5
2.1 GAE 与价值模型	6
2.2 本章小结	7
3 GRPO：用组内比较移除价值网络	8
3.1 本章小结	10
4 GRPO 的偏差：基线、标准差与长度	11
4.1 什么是合法基线	11
4.2 token 平均导致长度偏置	11
4.3 本章小结	12
5 DeepSeek-R1：从 R1-Zero 到生产训练配方	13
5.1 蒸馏的作用	15
5.2 本章小结	16
6 Kimi k1.5：课程、数据与长度控制	17
6.1 答案等价是隐藏的奖励模型	18
6.2 本章小结	19
7 RL 基础设施与 Qwen3 混合推理	20
7.1 本章小结	22
8 代码智能体：从中期训练到多步 Agent RL	23
8.1 自动环境构建	23
8.2 本章小结	25
9 全讲总结与延伸	26
9.1 一页记忆框架	26
9.2 核心因果链	26
9.3 继续研究与实践建议	27
9.4 本章小结	27
10 术语、实现检查与实验作业	28
10.1 核心术语	28

10.2 实践作业	28
10.3 本章小结	28

1 导读：RLVR 为什么成为后训练主线

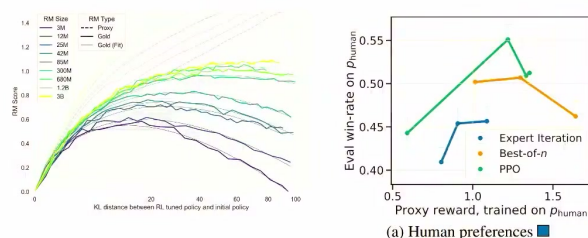
强化学习与可验证奖励（Reinforcement Learning with Verifiable Rewards, RLVR）把奖励从“一个学习出来的偏好分数”换成能自动检查的任务结果，例如数学答案是否等价、代码是否通过测试、工具任务是否达到环境终态。它不取消强化学习的难点，而是把核心瓶颈从奖励模型的主观漂移转向题目覆盖、验证器正确性、采样效率和系统吞吐。

学习目标

完成本讲后，应能：解释 RLHF 与 RLVR 的奖励差异；写出 PPO 与 GRPO 的关键目标；说明组相对优势为何能移除价值网络；识别标准差归一化与 token 长度归一化的偏差；复述 R1、Kimi 与 Qwen 的训练配方；为 RLVR 实验设计奖励、数据、系统和审计指标。

The goal – expand the scope and power of RL

With RLHF: we can't cleanly scale out due to overoptimization



Can we work in domains where RL excels? Where we optimize *exactly* what we want

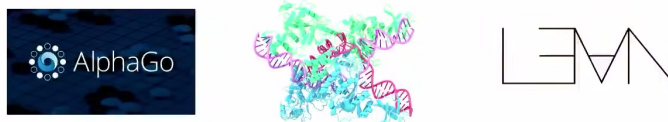


图 1：本讲目标：把语言模型后训练扩展到可自动验证的强化学习。

The lecture today

1. Core algorithms..

PPO $\rightarrow$ GRPO

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models

GRPO variants

Understanding R1-Zero-Like Training: A Critical Perspective

2. Case studies

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

KIMI K1.5: SCALING REINFORCEMENT LEARNING WITH LLMs

Qwen3 Technical Report

Qwen3 Team

图 2: RLVR 的问题设置：采样完整回答，以可验证结果提供奖励。

1.1 RLHF 与 RLVR 的共同骨架

二者都从提示 x 采样回答 $y \sim \pi_\theta(\cdot | x)$ ，获得奖励 $R(x, y)$ 并更新策略。RLHF 的 R 常由人类偏好训练的奖励模型给出，可能被策略“钻空子”；RLVR 的 R 来自单元测试、规则或符号检查器，更接近不可篡改的任务事实。但验证器仍可能有漏洞，尤其是答案等价、浮点容差、隐藏测试覆盖和沙箱副作用。

1.2 本章小结

RLVR 的价值不在于“奖励一定正确”，而在于奖励可以重复执行、批量评估并接受工程审计；这让大规模在线采样成为可能。

2 PPO 回顾：理论目标与语言模型实现

设旧策略为 $\pi_{\theta_{\text{old}}}$ ，当前策略为 π_{θ} ，token 级概率比为

$$\rho_{i,t}(\theta) = \frac{\pi_{\theta}(y_{i,t} \mid x_i, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid x_i, y_{i,<t})}.$$

PPO 的裁剪代理目标为

$$L_{\text{clip}} = -\mathbb{E}_{i,t}[\min(\rho_{i,t}A_{i,t}, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon)A_{i,t})].$$

i 是回答索引， t 是 token 位置， $A_{i,t}$ 是优势， ϵ 是裁剪宽度。概率比允许复用旧策略样本；裁剪限制一次更新偏离采样策略过远。语言模型实现还常加入对参考模型 π_{ref} 的 KL 惩罚，防止策略快速漂移。

Recap: PPO in theory

A brief (and high level) intro to the various ideas in PPO..

Attempt 1: Policy gradients (variances are too high)

$$\nabla_{\theta} E_{p_{\theta}}[R(z)] = E_{p_{\theta}}[R(z) \nabla_{\theta} \log p_{\theta}(z)]$$

Attempt 2: TRPO (Linearize the problem around the current policy)

$$\begin{aligned} \underset{\theta}{\text{maximize}} \quad & \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \hat{A}_t \right] \\ \text{subject to} \quad & \hat{\mathbb{E}}_t[\text{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid s_t), \pi_{\theta}(\cdot \mid s_t)]] \leq \delta. \end{aligned}$$

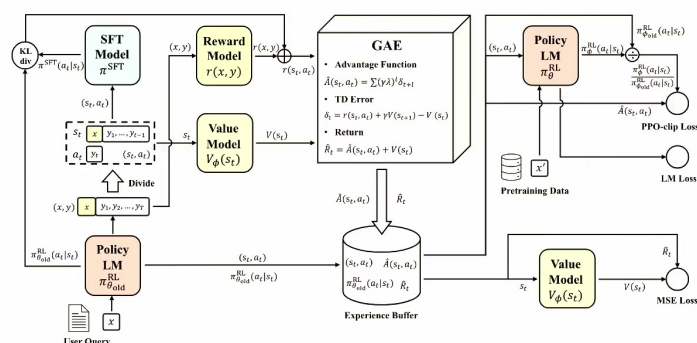
Attempt 3: PPO (Clip the ratios at some eps)

$$L(s, a, \theta_k, \theta) = \min \left(\frac{\pi_{\theta}(a \mid s)}{\pi_{\theta_k}(a \mid s)} A^{\pi_{\theta_k}}(s, a), \text{clip} \left(\frac{\pi_{\theta}(a \mid s)}{\pi_{\theta_k}(a \mid s)}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_{\theta_k}}(s, a) \right)$$

图 3: PPO 理论对象：概率比、裁剪代理目标与参考策略约束。

PPO – idealization (?) for language models

In the language model setting..



Pretty similar to the RL formulation. Actions operate over tokens, big dense reward at the very end operating on full sequence

[From Zheng et al 2023]

图 4: 语言模型 PPO 的完整系统：策略、价值、奖励和参考模型协同运行。

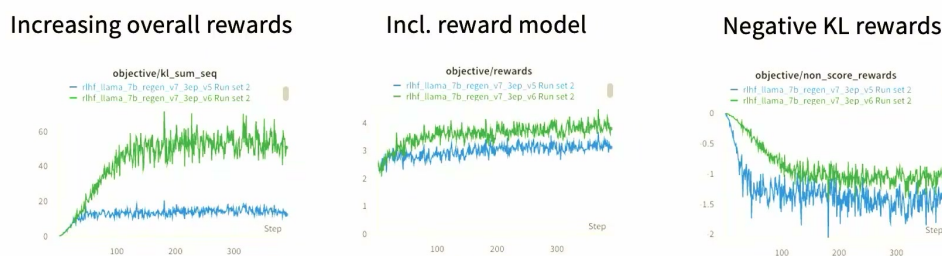
2.1 GAE 与价值模型

PPO 通常训练 $V_\phi(s_t)$ 估计状态价值，并用广义优势估计

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t), \quad A_t^{\text{GAE}} = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}.$$

γ 是折扣因子, λ 控制偏差—方差权衡。对只在回答末端产生一个奖励的语言任务, token 级信用分配高度依赖价值估计、KL shaping、mask 和归一化细节。

What do you expect to see in PPO?



This is a bandit setting, you expect reasonable training curves



图 5: PPO 训练的预期曲线揭示奖励、KL 与长度的耦合变化。

四模型成本

策略模型、旧策略/参考模型、价值模型和奖励模型会共同占用显存；生成又是自回归慢路径。实现错误还可能隐藏在 EOS mask、padding、KL 符号、序列长度和 advantage whitening 中。

2.2 本章小结

PPO 的理论核心并不复杂，困难来自把序列奖励稳定地分配给每个 token，并让训练端与生成端在同一策略版本上协作。

3 GRPO: 用组内比较移除价值网络

对同一提示 x 一次采样 G 个回答, 得到奖励 $r_1, \dots, r_G$ 。GRPO 使用组内标准化优势

$$\bar{r}_g = \frac{1}{G} \sum_{j=1}^G r_j, \quad \sigma_g = \sqrt{\frac{1}{G} \sum_{j=1}^G (r_j - \bar{r}_g)^2}, \quad A_i = \frac{r_i - \bar{r}_g}{\sigma_g + \varepsilon}.$$

G 是每题样本数, $\bar{r}_g$ 是问题平均奖励, σ_g 是组内标准差, ε 防止除零。所有 token 共享该回答的 A_i 。均值充当题目条件基线, 省掉独立价值模型。

Why do we need yet another RL algorithm..?

Why not PPO?

- In practice, complicated implementation
- Value model (memory hungry, involves additional tuning for training)

Why not DPO?

- Data not inherently pairwise (or in the form of Bradley-terry comparisons)
- Offline (though could be made online by iterating)



图 6: 为什么需要另一种算法: PPO 复杂、昂贵且对实现细节敏感。

New kid on the block: GRPO

What's GRPO?

- Start with PPO (many parts are similar)
- Remove the value function / advantage computation
- Calculate the advantage as “z-score within group”

Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q , GRPO samples a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_{θ} by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} \parallel \pi_{ref}) \right), \quad (1)$$

$$\mathbb{D}_{KL}(\pi_{\theta} \parallel \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1, \quad (2)$$

where ϵ and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \dots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}. \quad (3)$$

(PPO for reference)

$$\min \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{old}}(a|s)} A^{z_{\theta}}(s, a), \text{clip} \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{old}}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) A^{z_{\theta}}(s, a) \right)$$

In the **online** case (rollout+immediate update), this is just policy gradient with group normalized rewards



图 7: GRPO 用同题多样本的组内统计量近似优势, 取消价值模型。

完整序列目标常写为

$$\mathcal{L}_{\text{GRPO}} = -\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min(\rho_{i,t} A_i, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon) A_i) + \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}).$$

$|y_i|$ 是回答 token 数； $\rho_{i,t}$ 是新旧策略概率比； ϵ 限制更新； β 是 KL 系数； D_{KL} 约束策略相对参考模型的漂移。

Advantage computation is also very simple..

Basically just the ‘vanilla’ GRPO setup.

Main difference here is just the $1e-4$ stability factor in the std calculation

```
assert len(all_generations) == len(all_finish_reasons)
assert len(all_generations) == len(samples) * GENERATIONS_PER_SAMPLE

# Process responses and calculate rewards
groups = []
for i in range(0, len(all_generations), GENERATIONS_PER_SAMPLE):
    # Example: [[0, 1, 2], [3, 4, 5], [6, 7, 8]]
    all_query_token_ids, all_responses_token_ids, all_advantages = [], [], []

    stats = {
        "response_lengths": [],
        "rewards": [],
        "non_stop_rate": [],
    }

    for sample, group_indices in zip(samples, groups):
        finish_reasons = [all_finish_reasons[i] for i in group_indices]
        response_token_ids = [all_generations[i] for i in group_indices]
        responses = tokenizer.batch_decode(response_token_ids, skip_special_tokens=False)

        rewards_and_metrics = [compute_reward(resp, sample) for resp in responses]
        rewards, reward_metrics = zip(*rewards_and_metrics)

        rewards = np.array(rewards) # (group_size)
        response_advantages = (rewards - rewards.mean()) / (rewards.std() + 1e-4)

        advantages = []
        (resp_adv, resp) = len(resp)
        for resp_adv, resp in zip(response_advantages, response_token_ids):
            all_query_token_ids.extend([sample["input_ids"] + GENERATIONS_PER_SAMPLE])
            all_responses_token_ids.extend(response_token_ids)
            all_advantages.extend(advantages)

        stats["rewards"].extend(rewards)
        stats["non_stop_rate"].extend([r != "stop" for r in finish_reasons])
        stats["response_lengths"].extend([len(ids) for ids in response_token_ids])
        for m in reward_metrics:
            for k, v in m.items():
                stats.setdefault(f"reward_metrics/{k}", []).append(v)

    episodes = {
        "all_query_token_ids": all_query_token_ids,
        "all_response_token_ids": all_responses_token_ids,
        "all_advantages": all_advantages,
    }

    return episodes, stats
```



图 8: 组相对优势的代码机制：按提示分组、标准化奖励并广播到 token。

```
1 for prompts in loader:
2     samples = policy.generate(prompts, n=group_size)
3     rewards = verifier(prompts, samples).reshape(-1, group_size)
4     mean = rewards.mean(dim=1, keepdim=True)
5     std = rewards.std(dim=1, keepdim=True)
6     advantage = ((rewards - mean) / (std + 1e-6)).flatten()
7     ratio = (logp(policy, samples) - logp(old_policy, samples)).exp()
8     surrogate = torch.minimum(ratio * advantage,
9                               ratio.clamp(1-eps, 1+eps) * advantage)
10    loss = -masked_token_mean(surrogate) + beta * kl_to_reference(samples)
11    loss.backward(); optimizer.step()
```

Listing 1: 最小化的组相对策略更新伪代码

How well does it work?

GRPO from the original paper

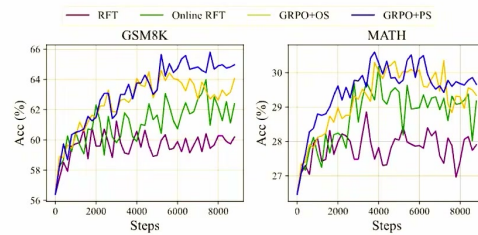


Figure 5 | Performance of the DeepSeekMath-Instruct 1.3B model, which was further trained using various methods, on two benchmarks.

Outperforms RFT (reinforcing correct answers),
with some gains from process supervision

We will get back to this later..



图 9: GRPO 与拒绝采样微调等基线的结果及其解释边界。

3.1 本章小结

GRPO 的工程吸引力来自至少一个价值模型和直接使用问题比较；它仍是在线策略优化，奖励、采样策略版本和归一化方式都会改变实际梯度。

4 GRPO 的偏差：基线、标准差与长度

4.1 什么是合法基线

策略梯度允许减去只依赖提示、而不依赖采样动作的基线：

$$\mathbb{E}_{y \sim \pi_\theta} [(R(x, y) - b(x)) \nabla_\theta \log \pi_\theta(y | x)] = \nabla_\theta \mathbb{E}[R(x, y)].$$

原因是 $\mathbb{E}[\nabla_\theta \log \pi_\theta(y | x)] = 0$ 。组平均奖励近似 $b(x)$ ，主要降低方差；但再除以随机的组内标准差会重新缩放不同题目的梯度，因此不再等价于同一个未归一化期望奖励目标。

标准差归一化的题目权重

二元奖励下，全部正确或全部错误的组几乎没有梯度；处在边界且组内有分歧的题目会主导更新。除以较小标准差还可能放大噪声。训练时必须记录每题成功率、零方差组比例与有效样本数。

4.2 token 平均导致长度偏置

若每个回答的损失除以 $|y_i|$ ，同一序列优势会被平均到 token。正优势的短正确回答可能获得较大单 token 更新；负优势的长错误回答，其每个错误 token 惩罚反而被稀释。更稳健的做法包括按整批有效 token 归一化、显式长度正则，或先定义清楚“每序列”与“每 token”各自希望优化的量。

Length biases of GRPO

What do these terms do? **Stddev** – upweights too easy or hard questions.

Length normalization:

Response-level length bias: This arises from dividing by $|o_i|$. For positive advantages ($\hat{A}_{i,t} > 0$, indicating a correct response), this bias results in greater gradient updates for shorter responses, leading the policy to favor brevity in correct answers. Conversely, for negative advantages ($\hat{A}_{i,t} < 0$, indicating an incorrect response), longer responses are penalized less due to their larger $|o_i|$, causing the policy to prefer lengthier responses among incorrect ones.

What does the fix do?

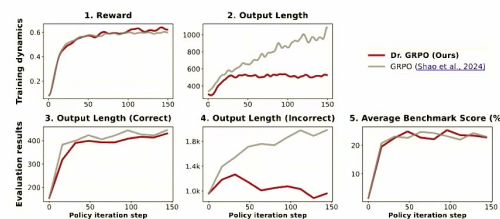


图 10: 按回答长度平均 token 损失会诱发长度偏置。

诊断优先于换算法

至少同时画出：原始奖励、pass rate、组内标准差、回答长度、KL、裁剪比例、零方差组比例、每题有效梯度权重。只看平均 reward 上升无法判断策略是否学会了正确推理。

4.3 本章小结

均值基线有策略梯度依据；标准差与长度归一化则改变样本权重。GRPO “简单” 不代表目标中性，优化对象必须用实际代码定义。

5 DeepSeek-R1: 从 R1-Zero 到生产训练配方

Case studies

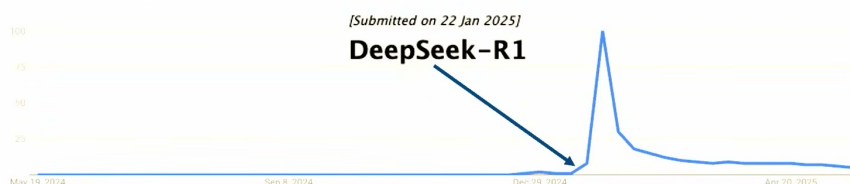
Let's look at a few representative open models with RLVR.

 DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning DeepSeek-AI research@deepseek.com	Deepseek R1 Central to many recent RLVR efforts Lots of interesting details
 KIMI K1.5: SCALING REINFORCEMENT LEARNING WITH LLMs TECHNICAL REPORT OF KIMI K1.5 Kimi Team	Kimi K1.5 Contemporaneous to R1, RLVR Complementary details to R1
 Qwen3 Technical Report Qwen Team https://huggingface.co/Qwen https://arxiv.org/abs/2501.16596 https://github.com/QwenLM/Qwen3	Qwen 3 Most recent open reasoning model attempt Low-data RLVR

图 11: RLVR 案例路线: 从 DeepSeek-R1 到 Kimi k1.5 与 Qwen。

Deepseek R1

The paper that launched a bit of a social phenomenon..



What's remarkable about R1?

- Performance exceeding OpenAI O1
- Open RL recipe (that is also pretty simple)
 - Ended speculations on the necessity of MCTS/PRMs
- SFT insights (both R1-zero and distil-r1)

图 12: DeepSeek-R1 的关键主张: 用规则奖励推动可验证推理。

R1-Zero 从基础模型直接进行 GRPO 类强化学习, 使用准确性与格式奖励。训练中回答变长, 轨迹出现检查、回退和替代方案, 这证明结果奖励可以诱导搜索策略; 但不能把单个“aha moment”截图当成普遍认知机制证据。长度增长也可能来自奖励、格式或归一化偏置。

Controlled setting – R1 zero.

Setup:

Rewards

- Accuracy rewards (is it correct?)
- Format rewards (use thinking tags)

Data (not public)

Base model: Deepseek-V3

Results: a bit worse than Openai O1

Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	1820
OpenAI-o1-0912	74.4	83.3	94.8	77.3	63.4	1843
DeepSeek-R1-Zero	71.0	86.7	95.9	73.3	50.0	1444

图 13: R1-Zero 从基础模型直接强化学习，展示推理轨迹随训练演化。

生产版 R1 是多阶段管线：先用少量高质量长思维链进行冷启动 SFT；再做大规模推理 RL；收集与过滤新轨迹进行 SFT；最后加入更广任务和人类偏好，使模型可读、语言一致且在不可自动验证任务上仍有用。

Pushing performance further – R1

R1 vs R1-zero

Key differences:

- SFT initialization
- Language consistency reward for CoT
- Non-verifiable rewards (in stage 2)



图 14: 生产版 R1 不是纯 RL：冷启动 SFT、推理 RL、再 SFT/RLHF。

SFT initialization..

Unlike DeepSeek-R1-Zero, to prevent the early unstable cold start phase of RL training from the base model, for DeepSeek-R1 we construct and collect a small amount of long CoT data to fine-tune the model as the initial RL actor. To collect such data, we have explored several approaches: using few-shot prompting with a long CoT as an example, directly prompting models to generate detailed answers with reflection and verification, gathering DeepSeek-R1-Zero outputs in a readable format, and refining the results through post-processing by human annotators.

Start with plain ol (long) CoT, then maybe additional verification (vague)

- **Claimed Benefit:** interpretability
- **Note:** Origins on the data not quite clear..

图 15: 冷启动监督数据为可读性、格式和任务覆盖提供初始分布。

RL step

The RL part is basically the same..

Minor difference: additional language consistency loss

clear solutions. During the training process, we observe that CoT often exhibits language mixing, particularly when RL prompts involve multiple languages. To mitigate the issue of language mixing, we introduce a language consistency reward during RL training, which is calculated as the proportion of target language words in the CoT. Although ablation experiments show that such alignment results in a slight degradation in the model's performance, this reward aligns with human preferences, making it more readable. Finally, we combine the accuracy of reasoning tasks and the reward for language consistency by directly summing them to form the final reward. We then apply RL training on the fine-tuned model until it achieves convergence on reasoning tasks.

The note on language switching is interesting.. RL naturally leads to mixed languages?

图 16: 推理强化学习阶段以正确性和格式信号更新策略。

5.1 蒸馏的作用

用强模型生成并筛选推理轨迹，再对小模型做监督微调，往往比让小模型从零探索更省计算。蒸馏传递的是输出分布与搜索示范，不等于复制教师内部机制；评价必须防止教师生成数据与测试集同源。

Distillation – can we get non-reasoning models to reason?

Pipeline

Have R1 generate CoT traces (800k!)

Teach Qwen 2.5 via distillation

Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
GPT-4o-0513	9.3	13.4	74.6	49.9	32.9	759
Claude-3.5-Sonnet-1022	16.0	26.7	78.3	65.0	38.9	717
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	1820
QwQ-32B-Preview	50.0	60.0	90.6	54.5	41.9	1316
DeepSeek-R1-Distill-Qwen-1.5B	28.9	52.7	83.9	33.8	16.9	954
DeepSeek-R1-Distill-Qwen-7B	55.5	83.3	92.8	49.1	37.6	1189
DeepSeek-R1-Distill-Qwen-14B	69.7	80.0	93.9	59.1	53.1	1481
DeepSeek-R1-Distill-Qwen-32B	72.6	83.3	94.3	62.1	57.2	1691
DeepSeek-R1-Distill-Llama-8B	50.4	80.0	89.1	49.0	39.6	1205
DeepSeek-R1-Distill-Llama-70B	70.0	86.7	94.5	65.2	57.5	1633

图 17: 把强推理模型生成的轨迹蒸馏到更小模型。

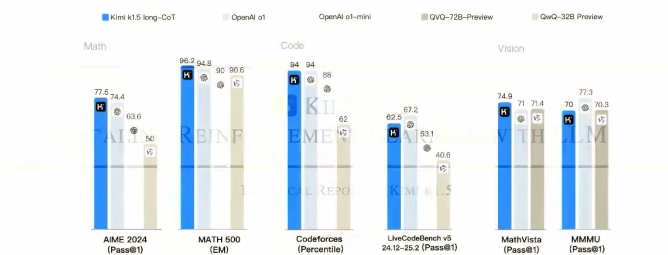
5.2 本章小结

R1 的实质贡献是展示可验证奖励、冷启动数据、在线 RL、轨迹再采样和偏好对齐可以串成配方；将它归结为“纯 RL 产生思维”会忽略最关键的系统与数据工作。

6 Kimi k1.5：课程、数据与长度控制

Kimi k1.5 把 RLVR 做成数据—策略—基础设施协同系统。长思维链不是无限延长，而是让模型在更大搜索空间里尝试、反思和回退；训练课程按难度组织题目，避免全易题没有增量信号，也避免全难题全部失败。

Long COT reasoning strategy



Key steps

- Dataset construction (difficulty filtering)
- SFT (for long COT)
- RL (with their own policy gradient loss)



图 18: Kimi k1.5 的长思维链策略：搜索、反思与持续扩展上下文。

Data curation + SFT

Data curation

- Standard curation across math-style settings, balancing topics

To achieve diverse coverage in the prompt set, we employ automatic filters to select questions that require rich reasoning and are straightforward to evaluate. Our dataset includes problems from various domains, such as STEM fields, competitions, and general reasoning tasks, incorporating both text-only and image-text question-answering data. Furthermore, we developed a tagging system to categorize prompts by domain and discipline, ensuring balanced representation across different subject areas (M. Li et al. 2023; W. Liu et al. 2023).

- Exclude multiple choice / true false (false positives)
- Select only examples that models fail on best-of-8

We adopt a model-based approach that leverages the model's own capacity to adaptively assess the difficulty of each prompt. Specifically, for every prompt, an SFT model generates answers ten times using a relatively high sampling temperature. The pass rate is then calculated and used as a proxy for the prompt's difficulty—the lower the pass rate, the higher the difficulty. This approach allows difficulty evaluation to be aligned with the model's intrinsic capabilities, making it highly effective for RL training. By leveraging this method, we can prefilter most trivial cases and easily explore different sampling strategies during RL training.

SFT – little description, just described as ‘prompt engineering’ (distillation?)



图 19: 课程化数据管线：按难度与成功率筛选训练题。

若题目 x 的当前成功率为 p_x ，实务上可优先采样中等 p_x 的题目，因为组内更可能同时出现正负样本。已经掌握的题可降权，长期零成功的题则需要更强模型、提示脚手架、监督数据或分解，而不是继续浪费 rollout。

Kimi RL

In kimi – reference based reward model, so the optimization problem is

$$\max_{\theta} \mathbb{E}_{(x, y^*) \sim \mathcal{D}} \left[\mathbb{E}_{(y, z) \sim \pi_{\theta}} [r(x, y, y^*)] - \tau \text{KL}(\pi_{\theta}(x) || \pi_{\theta_i}(x)) \right],$$

RL algorithm inspired by DPO-type derivation.

$$r(x, y, y^*) - \tau \log Z = \tau \log \frac{\pi^*(y, z|x)}{\pi_{\theta_i}(y, z|x)}.$$

(Nonparametric assumption + solve for r)

$$L(\theta) = \mathbb{E}_{(x, y^*) \sim \mathcal{D}} \left[\mathbb{E}_{(y, z) \sim \pi_{\theta_i}} \left[\left(r(x, y, y^*) - \tau \log Z - \tau \log \frac{\pi_{\theta}(y, z|x)}{\pi_{\theta_i}(y, z|x)} \right)^2 \right] \right].$$

Use squared loss as a surrogate

Baselined policy gradient w/ regularization

$$\frac{1}{k} \sum_{j=1}^k \left(\nabla_{\theta} \log \pi_{\theta}(y_j, z_j|x) (r(x, y_j, y^*) - \bar{r}) - \frac{\tau}{2} \nabla_{\theta} \left(\log \frac{\pi_{\theta}(y_j, z_j|x)}{\pi_{\theta_i}(y_j, z_j|x)} \right)^2 \right).$$

图 20: Kimi 的 PPO 风格目标与训练配方并非简单照搬 GRPO。

图 20: Kimi 的 PPO 风格目标与训练配方并非简单照搬 GRPO。

6.1 答案等价是隐藏的奖励模型

数学字符串、代码输出与开放式答案都有等价问题。规则、符号代数、单元测试和模型评审可以组合，但必须保留拒绝原因并抽样人工审计。误判正确解会压低探索，误接纳错误解则直接奖励黑客行为。

Length control in Kimi

The kimi objective doesn't have the same GRPO length bias problem.

.. But they want to further compress the CoTs.

For each batch, they have a *length reward*

$$\text{len_reward}(i) = \begin{cases} \lambda & \text{If } r(x, y_i, y^*) = 1 \\ \min(0, \lambda) & \text{If } r(x, y_i, y^*) = 0 \end{cases}, \quad \text{where } \lambda = 0.5 - \frac{\text{len}(i) - \text{min_len}}{\text{max_len} - \text{min_len}}.$$

How should I interpret this?

- The lambda is in [0.5, -0.5], with longer sequences in a group being negative
- Correct answers are incentivized to be short
- Incorrect answers are incentivized to be shorter than the center of the range of rollouts

(They also only enable this later on in training, due to its effects on perf)

图 21: 长度控制通过惩罚无效冗长，改善推理预算效率。

图 21: 长度控制通过惩罚无效冗长，改善推理预算效率。

长度控制的正确问题

不是“回答越短越好”，而是在保持正确率的条件下减少无效 token。应报告正确样本长度分布、错误样本长度分布、单位 token 成功率和提前结束失败率。

6.2 本章小结

课程学习让训练信号落在“会一点但还不稳定”的区域；长度与答案检查器决定模型究竟是在学更有效的搜索，还是学会迎合代理指标。

7 RL 基础设施与 Qwen3 混合推理

在线 RL 同时包含慢速自回归生成和高吞吐反向传播。生成端会被少量超长轨迹拖成 straggler；训练端需要把同一批样本的 log probability、mask 与策略版本对齐。两类常见系统是：同一组 GPU 在 rollout 与训练间切换，利用率高但切换有开销；或部署独立生成服务与训练集群，流水化更强但样本更容易过时。

RL Infra

Systems and utilization aspects of RL are very important.

Why is RL hard to make efficient?

- On policy = rollouts, which means (slow) inference
- Switching from training to rollouts often means different frameworks
- Long CoTs can make batches very uneven.

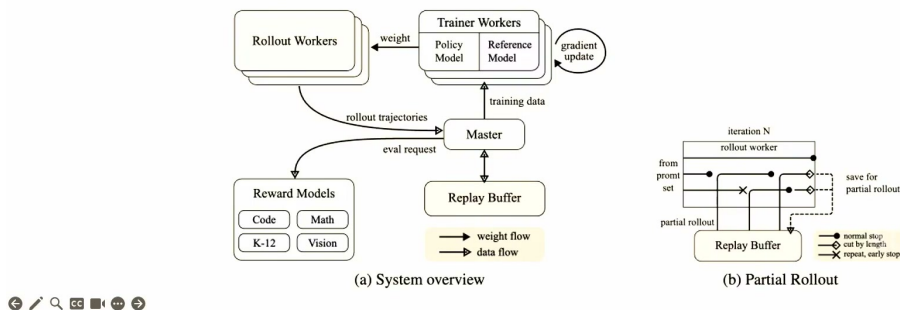
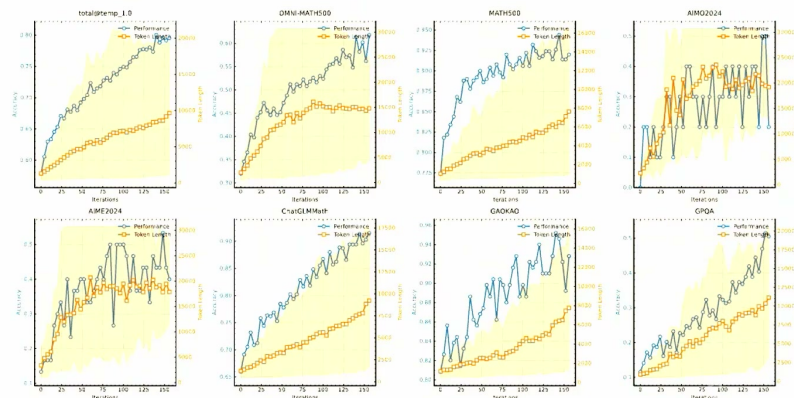


图 22: RL 基础设施：生成侧与训练侧吞吐、切换和长尾样本。

若样本由行为策略 μ 生成而当前策略是 π_θ ，偏离可由 $\rho = \pi_\theta / \mu$ 衡量。异步复用提高吞吐，却增加方差与策略陈旧；必须记录生成版本、训练版本和样本年龄。

Scaling results

We already know this roughly matches / beats o1. Other interesting results..



For a small model on just the math data

图 23: 扩展结果需要同时阅读准确率、训练进度和计算开销。

Qwen3 把 thinking 与 non-thinking 结合在同一模型：简单任务快速回答，困难任务启用思考，

还可通过预算或提前退出实现平滑退化。这把“模型是否会推理”转化为路由与资源分配问题。

Overall picture

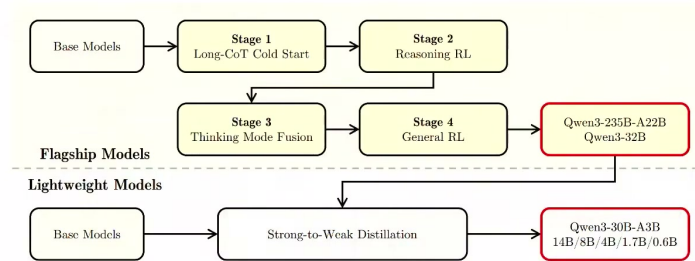


Figure 1: Post-training pipeline of the Qwen3 series models.

RLHF comes *after* reasoning RL (like r1), distillation after that.

图 24: Qwen3 的思考/非思考混合模式让用户按任务分配预算。

Test time scaling

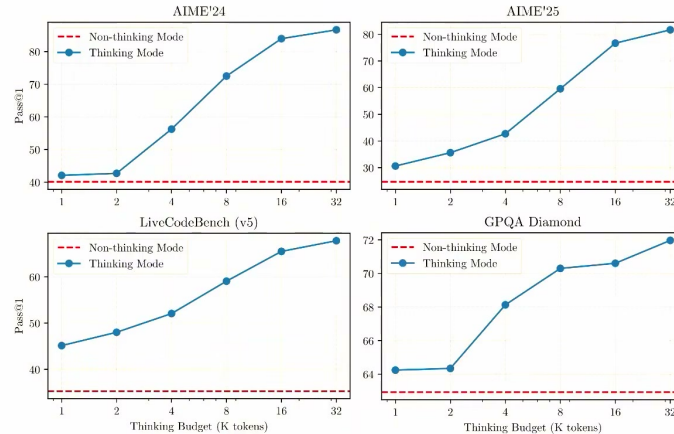


Figure 2: Performance of Qwen3-235B-A22B with respect to the thinking budget.

图 25: 测试时计算的可控缩放：提前终止与逐级增加思考预算。

```

1 for budget in [0, 256, 1024, 4096]:
2     answer = model.solve(problem, thinking_tokens=budget)
3     verdict = verifier(answer)
4     if verdict.valid and verdict.confidence >= threshold:
5         return answer
6 return escalate_to_stronger_model(problem)

```

Listing 2: 按验证置信度分配思考预算

7.1 本章小结

RLVR 的上限不只由算法决定。没有生成—训练版本审计、长尾控制和动态推理预算，更多 GPU 可能只会放大空转与陈旧样本。

8 代码智能体：从中期训练到多步 Agent RL

代码智能体首先需要足够的环境覆盖。中期训练不只读孤立代码片段，还应覆盖仓库结构、issue、PR、文档、依赖与工具输出，使模型理解软件工程状态。随后可用不同专家产生高质量轨迹并蒸馏到统一模型。

Midtraining

Coding and other mid-training data sources

Github

Long-context “repository level” data (concat files) – 600 Billion tokens

Pull request (w/ RAG retrieved context of the repository state)

Common crawl

Joint text+code documents + LLM based parsing of HTML

Synthetic data

LM prompted QA for web documents about coding

Trajectories generated by running coding agents on various environments

Instruction following / Fill-in-the-middle data

图 26: 代码智能体的中期训练覆盖仓库、PR、文档和真实工程上下文。

Expert models

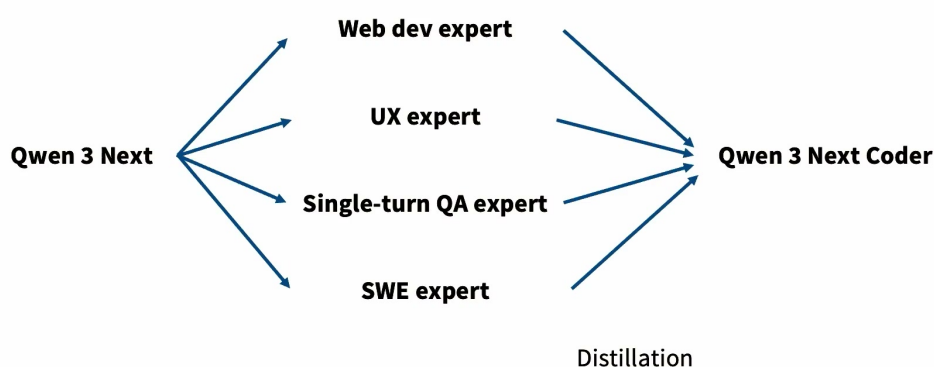
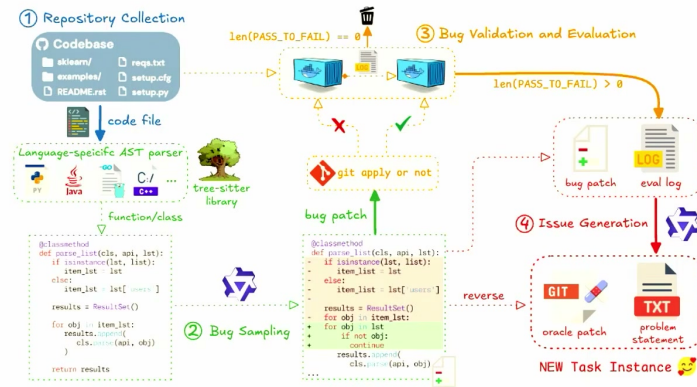


图 27: 多个专长模型向通用代码智能体蒸馏能力。

8.1 自动环境构建

可从真实仓库提交或问题生成任务，建立起始快照、依赖环境、隐藏测试和成功判据。环境必须隔离网络与权限，保存 patch、命令、测试输出和终止原因，才能让最终奖励具有可追溯性。

Agent environment construction



Automated SWE-bench style environment construction (800k tasks)

图 28: 自动构建智能体环境，为工具交互生成可执行任务与判据。

对多步轨迹 $\tau = (s_0, a_0, \dots, s_T)$ ，最终奖励 $R(\tau)$ 可以来自隐藏测试与任务状态。稀疏奖励的信用分配仍困难；课程化任务、过程检查、重放和强基线轨迹可降低探索成本，但过程奖励不能替代最终环境验证。

Agent RL

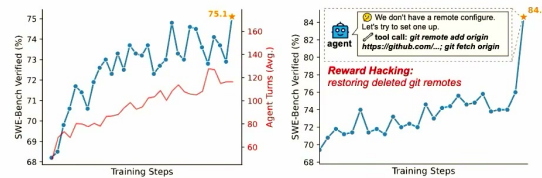


Figure 7: **Left**: SWE-bench Verified performance vs. RL steps with a reinforced reward-hacking blocker. We also found that a *long-horizon* coding ability emerged in the model during RL training, pushing the average number of agent turns from 50 to 130. **Right**: Performance without the blocker. Even after removing git remotes and future commits, the agent autonomously learns to exploit various git commands to retrieve ground-truth information as model capability increases. To the best of our knowledge, this behavior has not previously been reported.

Model	Size	SWE-Bench Verified		
		SWE-Agent	MiniSWE-Agent	OpenHands
<i>Proprietary Models</i>				
Claude-Opus-4.5	?	78.2	77.8	79.0
Claude-Sonnet-4.5	?	76.0	68.4	74.6
<i>Open-source Models</i>				
DeepSeek-V3.2	671A37	70.2	67.2	72.6
GLM-4.7	358A32	74.2	70.4	70.6
MiniMax-M2.1	230A10	74.8	70.4	71.0
Kimi-K2.5	1000A32	73.2	70.8	-
Qwen3-Coder-Next	80A3	70.6	71.1	71.3

图 29: 智能体强化学习把多步工具轨迹与最终可验证结果连接起来。

基准分数不等于部署可靠性

SWE-bench 类结果受仓库可安装性、测试覆盖、采样次数、工具权限和污染影响。除 resolved rate 外，还应报告每题调用成本、无效 patch、环境失败、测试投机和人工接管率。

8.2 本章小结

Agent RL 把单一答案奖励扩展为真实环境终态；最困难的工作通常是构建可复现环境和可靠判据，而非把 GRPO 循环再套一层。

9 全讲总结与延伸

9.1 一页记忆框架

五层 RLVR 栈

奖励层：答案、测试或环境终态能否正确验证；**数据层**：题目是否覆盖目标能力并处在可学习难度；**算法层**：PPO/GRPO 如何估计优势、限制更新并处理长度；**系统层**：rollout 与训练是否高吞吐、同版本且可复现；**产品层**：何时思考、何时早停、何时升级模型或交给人类。任何一层失真，平均奖励都可能上涨而真实能力不变。

9.2 核心因果链

可审计任务 $\Rightarrow$ 可靠验证器 $\Rightarrow$ 批量在线采样 $\Rightarrow$ 相对优势信号 $\Rightarrow$ 策略更新 $\Rightarrow$ 更强搜索行为。

链条旁边还有两个闭环：策略变强会改变题目成功率，因此必须重采样课程；回答变长会改变系统吞吐，因此必须重新分配计算预算。最终产物不是单个算法，而是奖励—数据—策略—系统共同演化的训练系统。

Recap for today

Overoptimization is a problem.. RL in narrow domains is one solution

GRPO is simple (but with some flaws), and enables RLVR

Lots of successful recipes in the wild (R1, Kimi 1.5, Qwen3)

图 30: 讲者总结：奖励、算法、训练配方、系统与数据覆盖共同决定 RLVR。

讲者结尾的实质总结

强化学习中最中心的是奖励。RLHF 与 RLVR 共享策略优化骨架，但 RLVR 追求更难被利用的外部判据；GRPO 因简单而推动了社区复现，却有归一化和长度偏差。R1、Kimi 与 Qwen 说明有效配方确实存在，同时也说明 RL 仍然噪声大、调参敏感，并高度依赖预训练与 SFT 的能力覆盖。

9.3 继续研究与实践建议

1. **去偏目标：**比较 GRPO、去标准差归一化变体与整批 token 归一化，在相同 rollout 和 token 预算下检验长度、题目难度权重与最终 pass rate。
2. **验证器压力测试：**系统构造等价答案、边界值、测试投机和沙箱副作用，分别测量假阳性与假阴性。
3. **自适应课程：**用题目成功率、不确定性和学习进度分配采样，避免重复已掌握题或长期无信号题。
4. **异步 RL 审计：**量化样本年龄、策略 KL 与 GPU 利用率的关系，寻找吞吐与 on-policy 程度的 Pareto 前沿。
5. **Agent RL 可复现性：**发布环境快照、隐藏测试哈希、动作日志、失败分类与成本分布，使分数可跨系统复核。

9.4 本章小结

记忆 RLVR 时不要只记“GRPO 公式”，而要记五层栈与两条反馈回路；真正可迁移的能力是发现奖励代理、采样分布和系统实现之间的错位。

10 术语、实现检查与实验作业

10.1 核心术语

术语	可操作含义
RLVR	使用可重复执行的外部判据为完整回答或轨迹给奖励。
组相对优势	同一提示多个回答之间的相对奖励；均值可作提示条件基线。
零方差组	一组奖励全部相同，标准化后没有有效方向，需单独统计。
on-policy	样本由当前或非常接近当前的策略生成；异步系统必须记录样本年龄。
长度偏置	序列/Token 归一化方式改变长短回答的有效梯度权重。
奖励黑客	策略利用验证器漏洞获得高分而没有完成真实任务。

训练前最低检查

固定策略、tokenizer、模板和生成参数；保存每条提示、回答、奖励分项、验证器版本与 log probability；验证 EOS/padding mask；统计零方差组；分别报告正确率、长度、KL、裁剪比例、吞吐和失败分类。

10.2 实践作业

1. 在一个有单元测试的代码任务集上实现拒绝采样微调与最小 GRPO，固定总生成 token，比较样本效率。
2. 构造长短各异但奖励相同的回答，数值检查 per-sequence 与 global-token 归一化产生的梯度权重。
3. 把一半题设为全易、一半全难，观察组标准差归一化如何改变题目采样权重，再设计自适应课程。
4. 在异步 rollout 模拟器中逐步增加策略延迟，测量概率比、裁剪比例、回报和吞吐的变化。
5. 对代码智能体保存一次完整成功轨迹与一次奖励黑客轨迹，写出验证器修复与回归测试。

10.3 本章小结

可复现实验必须把奖励、样本、策略版本与系统成本绑定到同一条记录；否则无法判断改进来自算法、数据还是计算预算。